

LINQ Last and LastOrDefault Methods in C#

In this article, I will discuss the **LINQ Last and LastOrDefault Methods in C#** with Examples using Method and Query Syntax. Please read our previous article, discussing the [LINQ First and FirstOrDefault Methods in C#](#) with Examples. The Last and LastOrDefault methods in LINQ are used to return the last element of a sequence. However, their behavior differs when the sequence is empty:

LINQ Last Method in C#:

The LINQ Last Method in C# returns the last element from a data source or a collection. If the data source or collection is empty, or if we specified a condition and with that condition, no matching element is found in the data source. The LINQ Last method will throw an **InvalidOperationException**. If the Data Source is Null, then it will throw **ArgumentNullException**. Two overloaded versions are available for the LINQ Last method, as shown in the image below.

```
public static TSource Last<TSource>(this IEnumerable<TSource> source);  
  
public static TSource Last<TSource>(this IEnumerable<TSource> source, Func<TSource,  
bool> predicate);
```

As you can see, the first overloaded version of the Last method does not take any parameter. It simply returns the last element from the data source, and if it is empty, it will throw an **InvalidOperationException**.

The second overloaded version of the Last method takes a predicate as a parameter. Using this predicate, we can specify a condition, and then it returns the last matching element that satisfies the specified condition. In this case, if no element satisfies the specified condition, it will throw **InvalidOperationException**.

Purpose, Behavior, and Usage of LINQ Last Method:

- **Purpose:** Returns the last element of a sequence.
- **Behavior:** If the sequence contains no elements, Last throws an **InvalidOperationException**.
- **Usage:** Use Last when you are confident that the sequence contains at least one element and that an empty sequence represents an exceptional scenario in the context of your application.

Example to Understand LINQ Last Method in C#:

Let us see an example of understanding LINQ's Last Method in C# with Method Syntax and Query Syntax. For a better understanding, please have a look at the following example. Here, we have created one data source, i.e., a collection containing integer numbers from 1 to 10. Our requirement is to fetch the first element from the data source, i.e., in the output, we want to display the value 10 as it is the last element in the data source. Here, we fetch the last element from the collection using the LINQ Last Method. No last operator is available in LINQ to write the Query Syntax. The example below shows how to combine the method and query syntax to write the code.

```
using System.Linq;  
using System;  
using System.Collections.Generic;
```

```

namespace LINQLastMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Fetching the Last Element from the Data Source using Last Method
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using Method Syntax
            int MethodSyntax = numbers.Last();
            //Query Syntax
            int QuerySyntax = (from num in numbers
                               select num).Last();
            //Printing the value returned by the Last Method
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}

```

Output: 10

Example to Understand LINQ Last Method, which takes Predicate as a Parameter in C#

We need to fetch the last element from the data source, divisible by 3. Here, we need to use the second overloaded version of the Last method to specify our condition. For a better understanding, please have a look at the following example. The following program uses the second overloaded version of the Last Method to return the last element from the data source divisible by 3. Many elements satisfy the given condition, but the Last method will return the last element that satisfies the given condition. In this case, the element will be 9. In the example below, I show how to use the second overloaded version of the LINQ Last method in C# using both Method and Query Syntax.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Fetching the Last Element from the Data Source which is Divisible by 3
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using Method Syntax
            int MethodSyntax = numbers.Last(num => num % 3 == 0);
            //Query Syntax
            int QuerySyntax = (from num in numbers
                               select num).Last(num => num % 3 == 0);
            //Printing the value returned by the Last Method
        }
    }
}

```

```

        Console.WriteLine(MethodSyntax);
        Console.ReadLine();
    }
}

```

Output: 9

What Happens When We Call the Last Method on Empty Data Source?

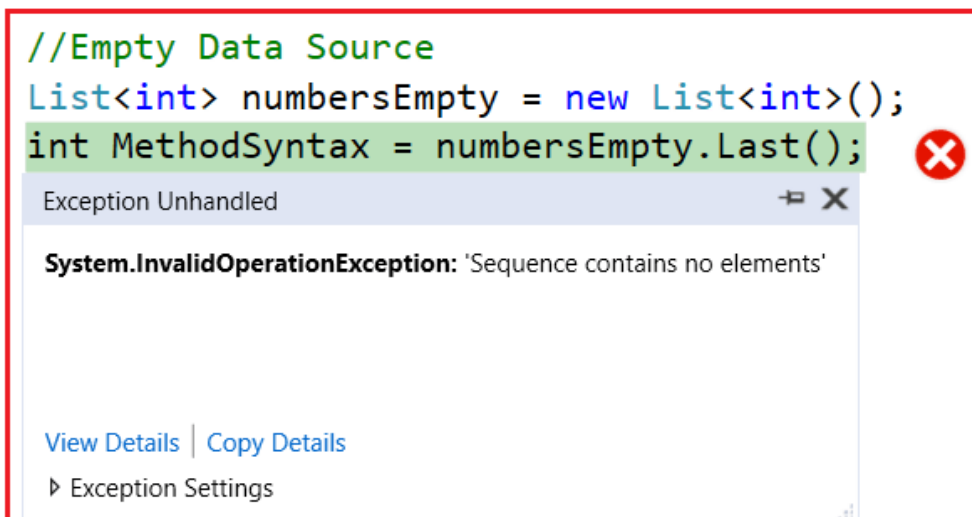
The LINQ Last Method will throw an `InvalidOperationException` whenever the data source is empty. For a better understanding, please have a look at the following example. In the example below, we call the Last method on an Empty Data Source; hence, we will get one Runtime Exception when we run the following code.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Empty Data Source
            List<int> numbersEmpty = new List<int>();
            int MethodSyntax = numbersEmpty.Last();
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}

```

Output:



What Happens If the Specified Condition in the Last Method Does Not Match with Any Element?

If the specified condition does not match with any element in the data source, then the LINQ Last Method will throw an **InvalidOperationException**. For a better understanding, please have a look at the following example. In the below example, we are calling the Last method and specifying the condition to fetch the last element, which is greater than 50. As you can see, the data source does not contain any element greater than 50; hence, running the below code will throw an **InvalidOperationException**.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Specified Condition Doesnot Return Any Element
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            int MethodSyntax = numbers.Last(num => num > 50);
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}
```

Output:

```
//Specified Condition Doesnot Return Any Element
List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int MethodSyntax = numbers.Last(num => num > 50);
```



Exception Unhandled



System.InvalidOperationException: 'Sequence contains no matching element'

[View Details](#) | [Copy Details](#)

▸ [Exception Settings](#)

Note: If you don't want that Invalid Operation Exception. Instead, if you want a default value based on the data type, you need to use the LINQ LastOrDefault method. For the reference types, the default value is NULL, and for the value types, the default value depends on the actual data type. For example, for numeric data types, it is 0. For string data types, it is null. And for the boolean data type, it is False.

LINQ LastOrDefault Method in C#:

The LINQ LastOrDefault Method in C# exactly does the same thing as the LINQ Last method does, except that this method does not throw the InvalidOperationException exception when the data source is empty or when the specified condition does not match with any element in the data source. In such cases, it will return the default value based on the data type of the data source. If the Data Source is Null, then like the Last Method, it will also throw ArgumentNullException. Like the Last method, two overloaded versions are available for the LastOrDefault method, as shown below.

```
public static TSource LastOrDefault<TSource>(this IEnumerable<TSource> source);  
  
public static TSource LastOrDefault<TSource>(this IEnumerable<TSource> source, Func<TSource,  
bool> predicate);
```

First Overloaded Version: It returns the last element of a sequence or a default value if the sequence contains no elements.

First Overloaded Version: It returns the last element of the sequence that satisfies the given condition or a default value if no such element is found in the data source or if the data source is empty.

Purpose, Behavior, and Usage of LINQ LastOrDefault Method:

- **Purpose:** Returns the last element of a sequence or the default value for the type of the elements if the sequence contains no elements.
- **Behavior:** If the sequence is empty, LastOrDefault does not throw an exception and instead returns the default value (for example, null for reference types, 0 for int, false for bool, etc.).
- **Usage:** Use LastOrDefault when an empty sequence is a common scenario and not exceptional. It allows you to safely handle cases where the sequence might be empty without the need for exception handling.

Example to Understand LINQ LastOrDefault Method in C#.

Let us see an example to Understand the LINQ LastOrDefault Method in C# with both Method and Query Syntax. Our requirement is to fetch the last Element from the Data Source using the LastOrDefault Method. For a better understanding, please have a look at the following example. Here, we have created one data source that contains integer numbers from 1 to 10. Then, we need to fetch the last element from the collection using the LastOrDefault method. There is no such operator called LastOrDefault available to write the Query Syntax. You can combine the method syntax and query syntax to write the code, as shown in the example below.

```
using System.Linq;  
using System;  
using System.Collections.Generic;  
namespace LINQLastOrDefaultMethodDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };  
            //Using Method Syntax  
            int MethodSyntax = numbers.LastOrDefault();  
        }  
    }  
}
```

```

        //Using Query Syntax
        int QuerySyntax = (from num in numbers
                           select num).LastOrDefault();
        Console.WriteLine(MethodSyntax);
        Console.ReadLine();
    }
}
}

```

Output: 10

Example to Understand LINQ LastOrDefault Method, which takes Predicate as a Parameter in C#

Now, we need to fetch the last element from the data source, which is less than 5. Here, we need to use the second overloaded version of the LastOrDefault method to specify our condition. For a better understanding, please have a look at the following example. The following program uses the second overloaded version of the LINQ LastOrDefault to return the last element from the data source, which is less than 5. Many elements satisfy the given condition, but the last element that satisfies the given condition is 4, which will be returned by the LastOrDefault method. In the example below, I show how to use the second overloaded version of the LINQ LastOrDefault method in C# using both Method and Query Syntax.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastOrDefaultMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Fetching the Last Element which is less than 5
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using Method Syntax
            int MethodSyntax = numbers.LastOrDefault(num => num < 5);
            //Using Query Syntax
            int QuerySyntax = (from num in numbers
                               select num).LastOrDefault(num => num < 5);
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}

```

Output: 4

What Happens when we call the LastOrDefault Method on an Empty Data Source or when the condition does not satisfy any element?

The LINQ LastOrDefault Method will not throw an InvalidOperationException whenever the data source is empty. Instead, it will return a default value based on the data type. Similarly, if no elements are found with the given condition, it will not throw any exception. Instead, it will return the default value based on the data type.

For a better understanding, please have a look at the following example. In the below example, we have created two data sources. The first data source contains no item; we are trying to fetch the last element here. The second data source contains 10 elements from the numbers 1 to 10. We are trying to retrieve the last element greater than 50 from this data source.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastOrDefaultMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Empty Data Source
            List<int> numbersEmpty = new List<int>();
            //Using Method Syntax
            int MethodSyntax1 = numbersEmpty.LastOrDefault();
            //Using Query Syntax
            int QuerySyntax1 = (from num in numbersEmpty
                               select num).LastOrDefault();
            Console.WriteLine(MethodSyntax1);
            //Specified condition doesnot return any element
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using Method Syntax
            int MethodSyntax2 = numbers.LastOrDefault(num => num > 50);
            //Using Query Syntax
            int QuerySyntax2 = (from num in numbers
                               select num).LastOrDefault(num => num > 50);
            Console.WriteLine(MethodSyntax2);
            Console.ReadLine();
        }
    }
}
```

Here, it will print the values as 0 and 0. This is because the data source contains integers. The default for integers is 0.

LINQ Last and LastOrDefault Method with Complex Type in C#:

Let us see an example of using LINQ Last and LastOrDefault Method with Complex Type in C#. For this, we are going to use the following Employee class. So, first, create a class file with the name **Employee.cs** and then copy and paste the following code into it. It is a simple class with 4 properties and one method to return a collection of employees.

```
using System.Collections.Generic;
```

```

namespace LINQLastOrDefaultMethodDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Salary { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", Salary = 10000, Gender = "Female"},
                new Employee { ID = 2, Name = "Priyanka", Salary = 20000, Gender = "Female"},
                new Employee { ID = 3, Name = "Anurag", Salary = 35000, Gender = "Male"},
                new Employee { ID = 4, Name = "Pranaya", Salary = 45000, Gender = "Male"},
                new Employee { ID = 5, Name = "Hina", Salary = 10000, Gender = "Female"},
                new Employee { ID = 6, Name = "Sambit", Salary = 30000, Gender = "Male"},
                new Employee { ID = 7, Name = "Mahesh", Salary = 35000, Gender = "Male"}
            };
        }
    }
}

```

Example to Understand LINQ Last Method with Complex Type in C#:

The following example shows how to use the LINQ Last method (both overloaded versions) with Complex Type. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastOrDefaultMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source
            List<Employee> listEmployees = Employee.GetAllEmployees();
            //Fetching the Last Employee from listEmployees Collection
            Employee Employee1 = listEmployees.Last();
            Console.WriteLine($"{Employee1.ID}, {Employee1.Name}, {Employee1.Salary}");
            //Fetch the Last Employee where the Gender is Male
            Employee Employee2 = listEmployees.Last(emp => emp.Gender == "Male");
        }
    }
}

```

```

        Console.WriteLine($"{Employee2.ID}, {Employee2.Name}, {Employee2.C
{Employee2.Salary}");
        //Fetch the Last Employee where the Salary is less than 30000
        Employee Employee3 = listEmployees.Last(emp => emp.Salary < 30000);
        Console.WriteLine($"{Employee3.ID}, {Employee3.Name}, {Employee3.C
{Employee3.Salary}");
        Console.ReadLine();
    }
}
}

```

Output:

```

7, Mahesh, Male, 35600
7, Mahesh, Male, 35600
5, Hina, Female, 10000

```

Example to Understand LINQ LastOrDefault Method with Complex Type in C#:

The following example shows how to use the LINQ LastOrDefault method (both overloaded versions) with Complex Type. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQLastOrDefaultMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source
            List<Employee> listEmployees = Employee.GetAllEmployees();
            //Fetching the Last Employee from listEmployees Collection
            Employee Employee1 = listEmployees.FirstOrDefault();
            Console.WriteLine($"{Employee1.ID}, {Employee1.Name}, {Employee1.C
{Employee1.Salary}");
            //Fetch the Last Employee where the Gender is Male
            Employee Employee2 = listEmployees.FirstOrDefault(emp => emp.Gende
"Male");
            Console.WriteLine($"{Employee2.ID}, {Employee2.Name}, {Employee2.C
{Employee2.Salary}");
            //Fetch the Last Employee where the Salary is less than 30000
            Employee Employee3 = listEmployees.First(emp => emp.Salary < 30000);
            Console.WriteLine($"{Employee3.ID}, {Employee3.Name}, {Employee3.C
{Employee3.Salary}");
            Console.ReadLine();
        }
    }
}

```

```
}  
}  
}
```

Output:

```
1, Preety, Female, 10000  
3, Anurag, Male, 35000  
1, Preety, Female, 10000
```

When to Use Last and LastOrDefault in LINQ?

Use the Last Method in LINQ

- **Use Case:** When you are certain the sequence contains at least one element, and an empty sequence is considered exceptional.
- **Appropriate Scenario:** After filtering a sequence where at least one element is expected to satisfy the condition, such as retrieving the most recent transaction from a list of transactions.
- **Example:** `var latestTransaction = transactions.Where(t => t.Date < DateTime.Now).Last();`

Use the LastOrDefault Method in LINQ

- **Use Case:** When the sequence may be empty, and you want to handle that case without an exception, usually by checking if the returned value is the default for the type.
- **Appropriate Scenario:** When querying data that may not exist, such as looking for a record that matches a condition that might not be met, you need to handle a 'not found' case gracefully.
- **Example:** `var lastSale = sales.LastOrDefault(s => s.Date < endOfFiscalYear);`

When Not to Use Them

- **Performance Consideration:** On non-indexed sequences (e.g., linked lists, non-list `IEnumerable`), using `Last` or `LastOrDefault` can be inefficient because the entire sequence must be enumerated to find the last element. In such cases, if you frequently access the last element, consider using a data structure with more efficient end access, like an array or a list.
- **Alternative Approach:** If you're working with a sequence that allows for efficient indexing from the end (such as an array or a `List<T>`), and you want to avoid the potential cost of enumerating the sequence, you might directly index the last element (e.g., `myList[myList.Count - 1]`).

Important Usage Notes:

- Use `LastOrDefault` to avoid exceptions when working with potentially empty sequences. The default value (null for reference types, 0 for numeric types, false for bool, etc.) can be checked to determine if the sequence was empty.
- Always ensure these methods align with your data's expected size and structure. If an operation assumes a non-empty sequence, but the sequence can be empty, this could lead to logical errors in your program.

- For performance-critical code, avoid using Last or LastOrDefault on large sequences that are not inherently indexed.

What is the Difference Between the Last and LastOrDefault Methods in LINQ?

Both Last and LastOrDefault methods in LINQ are used to return the last element from a data source. The Last and LastOrDefault methods in LINQ are used to retrieve the last element of a sequence, but they differ in how they handle sequences that contain no elements:

Last

- **Behavior:** It returns the last element of a sequence.
- **Exception Handling:** If the sequence contains no elements, Last will throw an InvalidOperationException.
- **Usage Consideration:** Use Last when you are certain that the sequence will contain at least one element, and an empty sequence would be considered an exceptional case in the logic of your application.

LastOrDefault

- **Behavior:** It returns the last element of a sequence or the default value for the type if the sequence contains no elements.
- **Exception Handling:** LastOrDefault does not throw an exception for an empty sequence; it returns the default value instead (e.g., null for reference types, 0 for integers).
- **Usage Consideration:** Use LastOrDefault when the sequence might be empty, and you want to handle this case without the overhead of exception handling. It allows for a more graceful degradation when dealing with potentially empty sequences.

So, choose between Last and LastOrDefault based on your expectations regarding the presence of a matching element and your preference for handling exceptional or empty sequence situations. Last is more suitable when you expect a match and want to handle exceptions, while LastOrDefault is useful when you want to handle default values gracefully when no match is found, or the sequence is empty.

In the next article, I will discuss the [LINQ Single and SingleOrDefault Methods](#) with Examples. In this article, I try to explain the LINQ Last and LastOrDefault Methods in C# with Examples. I hope you understand the need and use of these Last and LastOrDefault methods in LINQ.